

## Grupo 1: Desarrollo del Módulo de Gestión de Identidad y Accesos

### Descripción del trabajo:

El objetivo de este grupo es construir un sistema de autenticación robusto que supere el tradicional "usuario y contraseña". Los estudiantes deben centrarse en garantizar que quien accede es quien dice ser y que su sesión se mantiene segura durante todo el ciclo de vida de la conexión. Este trabajo aborda directamente la gestión de sesiones y contraseñas.

### Requerimientos a implementar:

- ✓ **Gestión de Credenciales:** Implementar reglas estrictas para la creación de contraseñas (uso obligatorio de mayúsculas, minúsculas, números y caracteres especiales).
- ✓ **Doble Factor de Autenticación (2FA):** Al loguearse correctamente, el sistema debe exigir un segundo paso, simulando el envío de un código de verificación (OTP) al correo o móvil.
- ✓ **Control de Concurrencia (Acceso Único):** El sistema no debe permitir que el mismo usuario tenga sesiones abiertas simultáneamente en diferentes navegadores o dispositivos. Si se abre una segunda, la primera debe cerrarse.
- ✓ **Tokens con Caducidad:** Uso de tokens de sesión con tiempos de vida cortos y mecanismos de renovación segura.

### ¿Cómo realizarlo?

Deben utilizar librerías de validación (expresiones regulares) en el frontend para la complejidad de la clave. Para la concurrencia, en el backend (base de datos) se debe mantener un registro del ID\_Sesión actual; si llega una petición con un ID antiguo, se rechaza. Para el 2FA, pueden usar librerías de generación de códigos aleatorios temporales.

## Grupo 2: Blindaje del Lado del Cliente y Protección de Datos Locales

### Descripción del trabajo:

Este grupo se enfocará en la seguridad "visible" y en el navegador del usuario. Su misión es dificultar que un atacante extraiga información sensible almacenada temporalmente o entienda la lógica del negocio mediante la inspección del navegador. Esto cubre la seguridad del lado del cliente.

### Requerimientos a implementar:

- ✓ **Cifrado de Variables de Sesión:** La información almacenada en el navegador (Almacenamiento Local o de Sesión) nunca debe estar en texto plano; debe estar encriptada.
- ✓ **Protección Visual y de Código:** Implementar scripts para desactivar el clic derecho (menú contextual) y dificultar la inspección rápida del código fuente.
- ✓ **Sesión Temporizada:** Implementar un monitor de inactividad que cierre la sesión automáticamente si el usuario no mueve el mouse o teclea tras un tiempo definido.
- ✓ **Ofuscación:** El código JavaScript entregado al cliente debe estar minificado y ofuscado para que no sea legible humanamente.

¿Cómo realizarlo?

Utilizar librerías de criptografía estándar (como AES) para cifrar los datos antes de guardarlos en el navegador. Usar EventListeners de JavaScript para detectar la inactividad y reiniciar un temporizador de cierre de sesión. Para el clic derecho, capturar y prevenir el evento contextmenu.

### Grupo 3: Arquitectura de Servicios Web Seguros y Control de Acceso

Descripción del trabajo:

Este equipo debe proteger la "puerta trasera" de la aplicación. Se centrarán en configurar cómo el servidor acepta y procesa las peticiones externas, asegurando que solo los clientes autorizados puedan consumir los datos. Se relaciona con la construcción de software seguro.

#### Requerimientos a implementar:

- ✓ **Listas Blancas de Acceso:** Permitir el consumo de servicios solo desde direcciones IP específicas o dominios controlados, rechazando el resto.
- ✓ **Consumo Exclusivo desde el Frontend:** Configurar las políticas de intercambio de recursos (CORS) para que la API solo responda a la aplicación web oficial, no a peticiones de otros orígenes.
- ✓ **Servicios vía POST:** Diseñar los servicios críticos utilizando únicamente el verbo HTTP POST para evitar que los parámetros sensibles queden visibles en la URL o en el historial del navegador.
- ✓ **Restricción por DNS/IP:** Validar el origen de la petición antes de procesar cualquier lógica de negocio.

¿Cómo realizarlo?

Configurar los encabezados HTTP en el servidor (backend) para definir estrictamente el Access-Control-Allow-Origin. Implementar Middleware (software intermedio) que verifique la IP del cliente contra una base de datos o archivo de configuración de direcciones permitidas antes de dar paso al controlador.

### Grupo 4: Pruebas de Carga y Mecanismos de Defensa contra Ataques Automatizados

Descripción del trabajo:

Este grupo simulará el rol de atacantes y defensores simultáneamente. Su objetivo es proteger el servidor contra el abuso y la saturación. Este tema es importante para validar la seguridad del software bajo condiciones extremas, tocando puntos de validación y construcción.

#### Requerimientos a implementar:

- ✓ **Limitación de Tasa (Rate Limiting):** Controlar el número de peticiones que un usuario o IP puede hacer en un segundo/minuto.
- ✓ **Pruebas con JMeter:** Deben diseñar un script de pruebas con la herramienta JMeter que intente saturar el servicio.
- ✓ **Bloqueo Automático:** Si el sistema detecta un exceso de peticiones (según la prueba de JMeter), debe bloquear temporalmente esa dirección IP o usuario.

¿Cómo realizarlo?

En el backend, implementar un algoritmo de "cubeta de fichas" (Token Bucket) o usar librerías de limitación de tasa disponibles en el lenguaje de programación elegido. Deben configurar JMeter para lanzar hilos concurrentes y demostrar cómo el servidor empieza a responder con códigos de error (como 429 Too Many Requests) cuando se supera el límite.

### Grupo 5: Prevención de Inyecciones de Código y Documentación de Seguridad

Descripción del trabajo:

Este grupo tiene la responsabilidad de integrar la seguridad técnica con la formalidad de la ingeniería de software. Deben asegurar que los datos ingresados no rompan la base de datos ni ejecuten scripts maliciosos, y documentar todo el proceso como una "buena práctica". Esto une la prevención de Inyección SQL y XSS con las buenas prácticas y documentación.

#### Requerimientos a implementar:

- ✓ **Sanitización de Entradas:** Implementar filtros que limpien cualquier dato ingresado en formularios para evitar Inyección SQL y Cross-site Scripting (XSS).
- ✓ **Manual de Buenas Prácticas:** Redactar un documento formal que explique las políticas de seguridad implementadas en el proyecto (ej. "Política de longitud de contraseñas", "Política de manejo de sesiones").
- ✓ **Validación de Tipos de Datos:** Asegurar que si un campo pide números, el *backend* rechace cualquier cosa que no sea un número estrictamente.

¿Cómo realizarlo?

Deben utilizar "Consultas Preparadas" (Prepared Statements) obligatoriamente para todas las interacciones con la base de datos. Para XSS, utilizar funciones de codificación de salida (convertir caracteres especiales en entidades HTML). El documento debe seguir un formato estándar de ingeniería, explicando el problema, la solución adoptada y la evidencia del funcionamiento.